



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 6

Detailed Design & Design Principles



Ursinus
COLLEGE

Team Standup Meetings



Last Week

Software Architecture: Engineering Structure Under Constraint



What is Detailed Design?

The specification of internal components (classes, services, methods, data schemas) and how they collaborate to satisfy requirements.



Why Detailed Design Matters

- Controls complexity
- Enables testability
- Improves maintainability
- Prevents requirement leakage
- Supports traceability



The Core Question

For each requirement:
What isolated design unit satisfies this requirement?



Requirement → Design Mapping Example

Requirement R1: “Users must reset password via email”

What does this actually require the system to do?



Decomposing the Requirement

What Capabilities Are Introduced?

- Generate secure reset token
- Store token with expiration
- Send reset email
- Validate token
- Update password securely
- Prevent token reuse



Option 1: Add to AuthenticationService

AuthenticationService might already include:

- login(user,password)
- logout(user)
- validateToken(token)

We could add:

- requestPasswordReset(email)
- resetPassword(token, newPassword)



Option 2: Dedicated PasswordResetService

PasswordResetService:

- requestReset(email)
- validateToken(token)
- updatePassword(token, newPassword)



Option 3: Strategy Pattern (Future-Oriented)

PasswordResetStrategy (interface, not class):

- executeReset()

Implementations:

- EmailResetStrategy
- SMSResetStrategy



Supporting Schema

PasswordResetToken Table

Field	Purpose
token_id	Unique token
user_id	Owner
expiration_time	Security
used_flag	Prevent reuse



What NOT to do

Poor Design Choices

- ✗ Put logic in UserController
- ✗ Mix with profile update logic
- ✗ Directly manipulate database in controller
- ✗ Create a “UserManager” “God class”



Signs of Bad Detailed Design

- One class satisfies 5+ unrelated requirements
- Methods exceed one screen
- No clear pre/post conditions
- No mapping to requirements
- No data ownership clarity



What Do We Mean By Modular?

Modular:

- Self-contained
- Clear interface
- Individual responsibility

Abstract:

- Depends on interfaces, not implementations

Isolated:

- Limited side effects
- Minimal dependencies



Why Design Principles Exist

Software complexity increases over time.

Design principles:

- Reduce entropy
- Control change
- Improve testability
- Protect maintainability



Design Principle 1: Single Responsibility Principle (SRP)

A module should have one reason to change.

Not:

“It does one thing” ✗

But:

“It changes for one reason” ✓

A module should be responsible to one actor or concern in the system.

The goal is to reduce ripple effects when requirements change.



Example of a class that violates SRP

UserService

- login()
- resetPassword()
- updateProfile()
- sendMarketingEmail()
- generateAnalyticsReport()



SRP in Microservices

In a microservices architecture:

One service = one business capability.

Warning sign:

A service implements 4+ requirement keys.



Design Principle 2: High Cohesion

Cohesion = How strongly related the responsibilities are within a module.

High Cohesion:

- Focused ✓
- Understandable ✓
- Maintainable ✓

Low Cohesion

- Random Functionality ✗
- Mixed Concerns ✗



Cohesion Example:

High Cohesion:

AuthService:

- login()
- validateToken()
- logout()

Low Cohesion:

AuthService:

- login()
- sendPromotionalEmail()
- exportUserDataCSV()



Design Principle 3: Low Coupling

Coupling = How much one module depends on another.

Low Coupling:

- Change in A does not break B ✓
- Clear boundaries ✓
- Replaceable components ✓

High Coupling:

- Hidden dependencies ✗
- Ripple effects ✗
- Fragile systems ✗



Coupling Example:

High Coupling (Bad): ✗

OrderService directly queries raw SQL in 12 places

Now if schema changes: 12 updates

Low Coupling (Better) ✓

OrderService → OrderRepository → Database

Only the repository needs to change if schema changes



Coupling Types (Briefly)

Common types of coupling:

- Content coupling (worst)
- Common/global coupling
- Control coupling
- Data coupling (acceptable)

We aim for:

Data coupling through clear interfaces

Good systems minimize the knowledge modules have about each other.



Design Principle 4: Abstraction

Expose *what*

Hide *how*

Abstraction creates:

- Flexibility
- Testability
- Replaceability



Interface-Based Design

Depend on abstractions, not concrete implementations.

Example:

PaymentProcessor (interface)
+ processPayment()

Concrete classes implement it.



Design Principle 5: Separation of Concerns

Different concerns should be handled by different modules.

Examples of concerns:

- Business logic
- Data access
- UI
- Validation
- Logging
- Security



Design Principle 6: Design for Testability

A well-designed module:

- Has clear inputs
- Has predictable outputs
- Has explicit preconditions
- Has measurable postconditions



When Principles Conflict

Design is about trade-offs.

Too much abstraction → complexity

Too many microservices → network coupling

Too many small classes → fragmentation

Balance matters.



Applying Principles to Your Assignments

For EACH requirement, Ask yourself:

1. Does one module satisfy it?
2. Does that module have one reason to change?
3. Is it highly cohesive?
4. Is coupling minimal?
5. Are pre/post conditions clear?



Pre- & Post-Conditions

Preconditions are conditions that must be true before execution.

Examples:

- userID exists
- Password meets complexity requirements
- Token not expired

Postconditions are guaranteed outcomes after execution.

- Password updated
- Audit log written
- Token invalidated

This matters because Pre/Post-Conditions become unit tests, acceptance tests, and formal verification.



Appropriate Level of Method Detail

Include:

- Method Name ✓
- Input parameters ✓
- Return type ✓
- Preconditions ✓
- Postconditions ✓

Exclude:

- Full algorithm ✗
- Implementation details ✗
- Code ✗



What is UML?

UML = Unified Modeling Language

Developed and introduced in the mid-1990s by Grady Booch, Jim Rumbaugh, and Ivar Jacobson at Rational Software

A standardized visual language for describing software systems.



Why Use UML?

- Makes structure visible
- Improves communication
- Clarifies responsibilities
- Reveals coupling
- Exposes design flaws early



The Only UML Diagrams You Need for Your Assignment

We will use:

- Class Diagrams
- Sequence Diagrams (optional but encouraged)
- Component Diagrams (for microservices)

That's it.



What is a Class Diagram?

Shows:

- Classes (or services)
- Methods
- Attributes
- Relationships between them

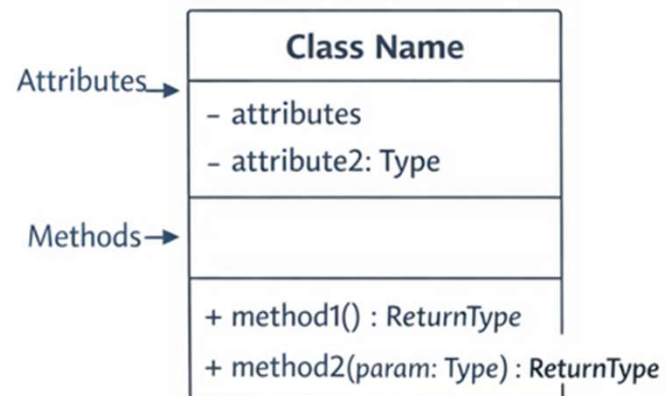
This is the primary diagram for your assignment.



Basic Class Diagram Box Structure

A class diagram box has 3 sections: ClassName, attributes, and methods:

Class Box Structure





Example: AuthenticationService

Example: AuthenticationService

AuthenticationService
- userRepository: UserRepository - tokenService: TokenService
+ login(email: String, password: String) ; + resetPassword(email: String) ; void + validateToken(token: String) ; boolean



Relationships in Class Diagrams

We will use only 4 relationships:

1. Association

- Meaning: “Has-a” (Structural relationship, implies ownership)
- Draw as a solid line (Arrow optional depending on context)
- ClassA —> ClassB

2. Dependency

- Meaning “Uses-a” (Temporary relationship, No ownership)
- Draw as a dashed arrow
- Points from the dependent class to the class it depends on
- ClassA ----> ClassB



Relationships in Class Diagrams (continued)

3. Inheritance (Generalization)

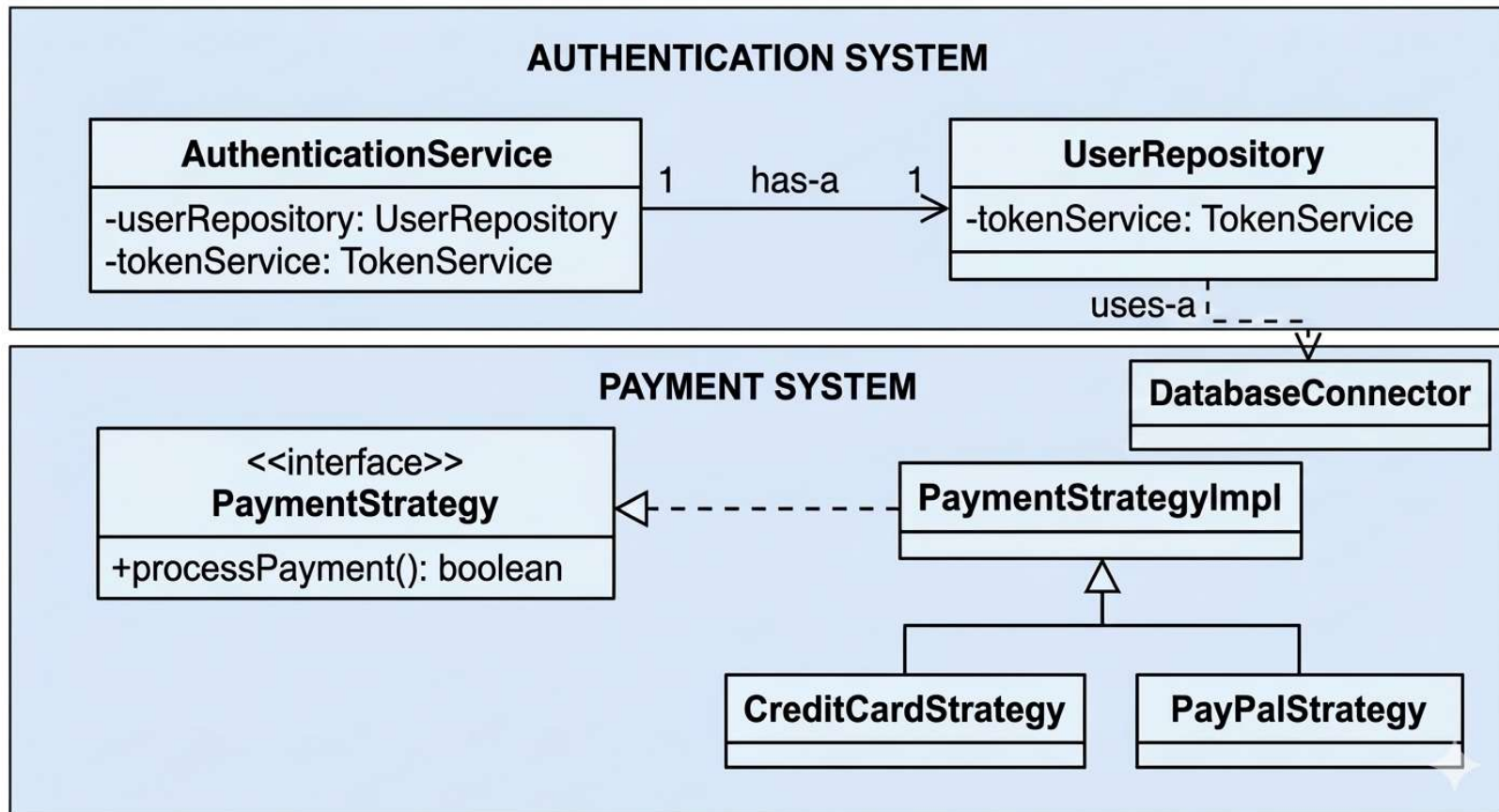
- Meaning: “Is-a”
- Draw as a solid line, hollow triangle arrowhead
- Arrow points to parent (superclass)
- AdminUser —▷ User

4. Interface Implementation (Realization)

- Meaning “Implements”
- Dashed line with hollow triangle arrowhead
- Arrow points to interface
- PasswordResetService - - - ▷ IResetStrategy



Common Relationships in UML Class Diagrams





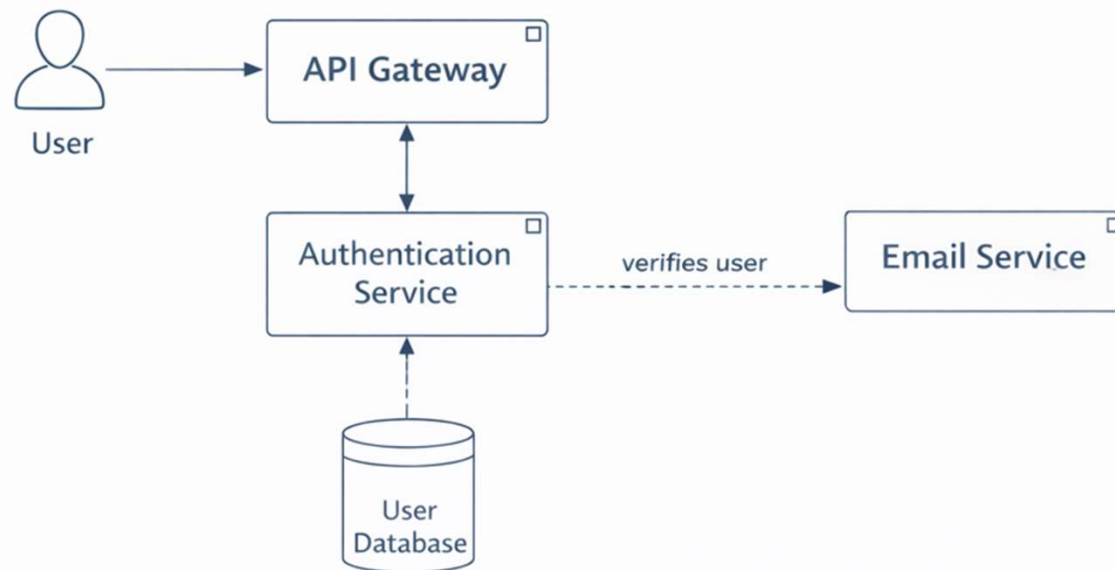
Component Diagrams (For Microservices)

Use these if your project includes

- Multiple microservices
- External APIs
- Databases
- Cloud storage



Component Diagram Example





Sequence Diagrams (Optional But Recommended)

Sequence diagrams show interaction over time

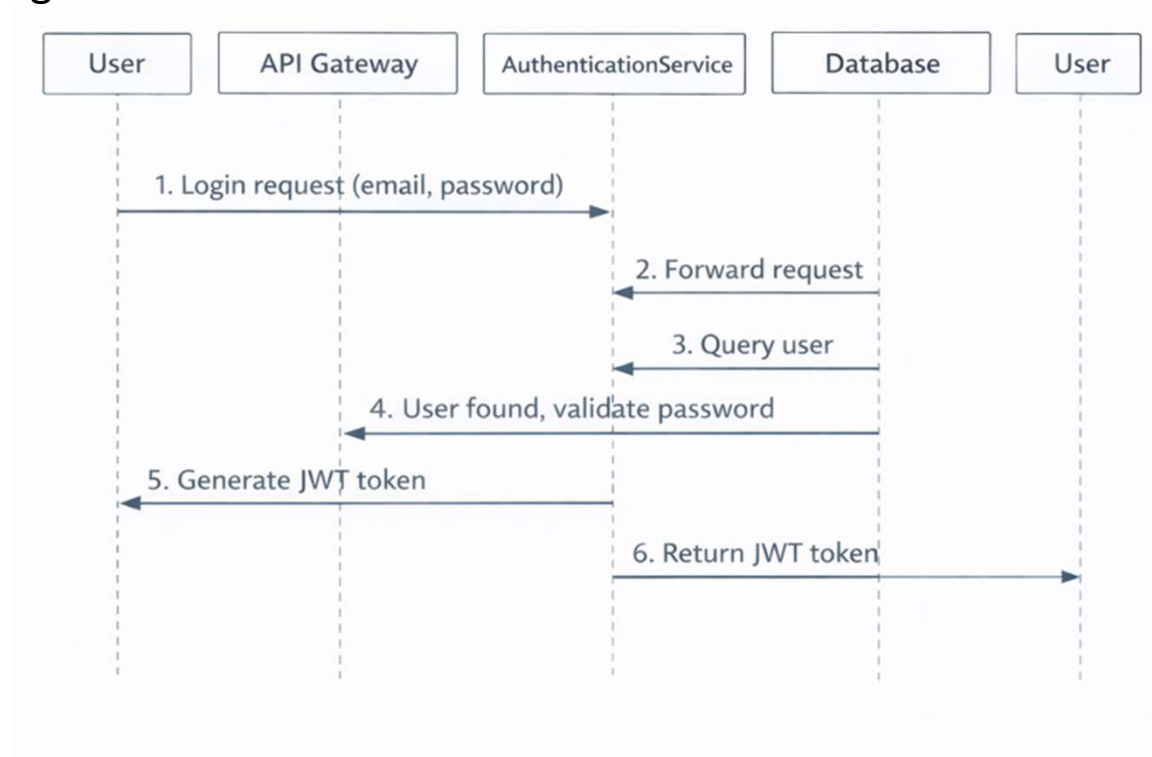
They clarify flow, reveal excessive coupling, and help define pre/post conditions.

But they are optional for your assignment.



Sequence Diagrams (Optional But Recommended)

Example: Login Flow:





Code Design vs Data Design

Software systems have two fundamental structures:

Code Structure:

- Classes, services, and interactions
- Represented with **UML Diagrams**

Data Structure:

- Persistent data and relationships
- Represented with **database schemas**

Both must trace back to requirements



Why Schema is Part of Design

Data model = Domain model (hopefully).

Bad schema creates:

- Tight coupling
- Poor cohesion
- Scaling issues

Every schema element should support a requirement.

If a table supports 6 unrelated requirements, consider redesign.



Minimal Schema Requirements

Show:

- Table Name
- Fields
- Data types (keep it simple)
- Primary Key
- Foreign Keys

Example:

Users

- user_id (string) (PK)
- email (string)
- password_hash (string)

PasswordResetTokens

- token_id (string) (PK)
- user_id (string) (FK)
- expiration_time (datetime)



UML Reference Resources

Official UML Overview:

<https://www.uml.org/>

UML Class Diagram Tutorial:

<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-class-diagram/>

UML Cheat Sheets:

<https://modeling-languages.com/best-uml-cheatsheets-and-reference-guides/>



Requirement Flowchart

For each requirement include a **simple flowchart** showing the execution path.

Show:

- Entry point
- Services/components touched
- Decision points (if relevant)
- Data store interactions

Keep it:

- Under ~7–10 boxes
- Focused only on that requirement



Requirement Trace Flow

For each requirement, show the path through services/components.
This will be visualized as a **simple flowchart, separate from** your UML diagrams.

Example:

User submits password reset request



API Gateway receives request



AuthService checks if user exists



[Decision] User found?

/ Yes

\ No



Generate reset token Return error



Store token in TokenStore (DB)



EmailService sends reset email



Assignment Expectations

What I Expect (Minimal Standard)

For Each Requirement include:

Design Mapping

- Identify the class, module, or service responsible
- Reference the **requirement key**

UML Design

- One **UML diagram** showing the relevant components
- Dependencies or relationships
- Methods listed inside the class/service, **including their preconditions or postconditions**

Data Design

- **Schema** for any required **database** or **storage**

Requirements Trace

- A **flowchart** showing the execution path
- Entry point -> services -> data stores -> outcome



Assignment Expectations

What I Do **NOT** Expect

- Perfect UML compliance
- Every private variable shown
- Complex multiplicities
- Overly detailed diagrams

Clarity > Perfection



Sample Requirement Package

Requirement Summary:

Requirement Key: R1

Title: User Login

Description: The System shall allow a registered user to authenticate using email and password and receive an authentication token upon successful login.



Sample Requirement Package

Design Element That Satisfies This Requirement:

We will satisfy R1 using:

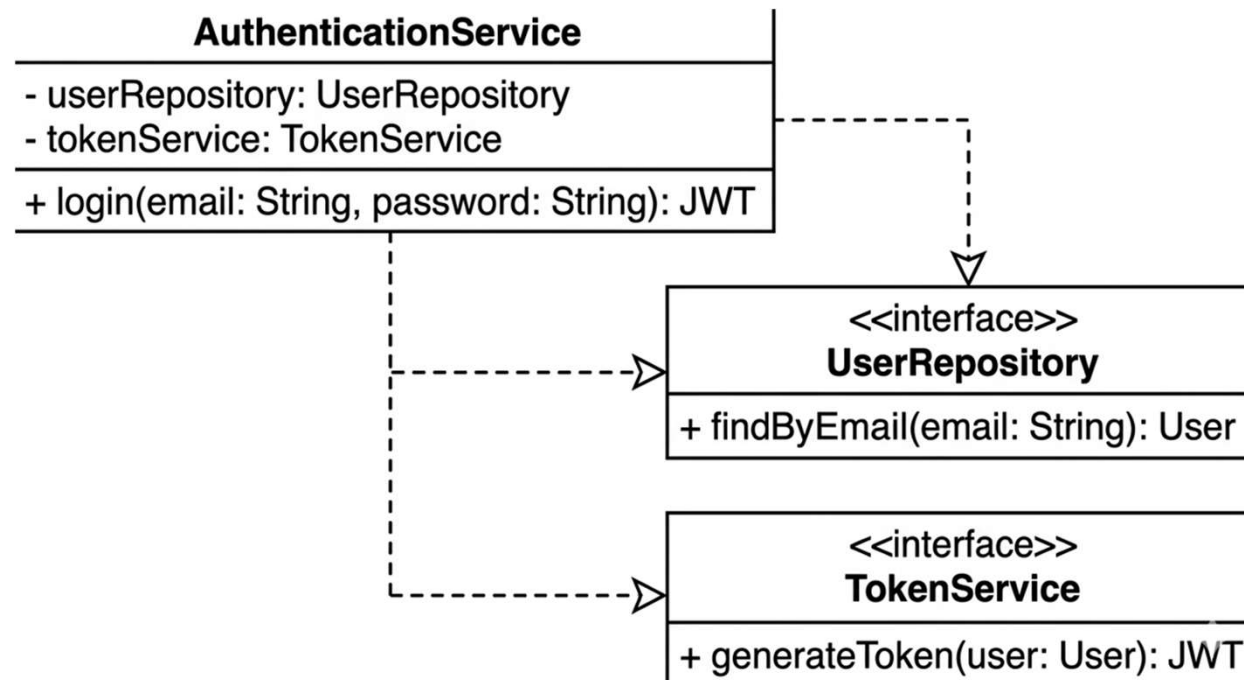
- **Authentication Service** (service layer)
- **User Repository** (data access abstraction)
- **Token Service** (token generation abstraction)

This isolates authentication responsibility



Sample Requirement Package

UML Class Diagram





Sample Requirement Package

Method Specification:

Method: login (email, password): (JWT)

- **Inputs**
 - email: String
 - password: String
- **Preconditions**
 - email is not null or empty
 - password is not null or empty
 - User exists in the system
 - Account is not locked
- **Postconditions (Success case)**
 - Password hash matches stored hash
 - A JWT token is generated
 - Login event is logged
 - JWT is returned to caller
- **Postconditions (Failure Case)**
 - No token generated
 - Failed login attempt recorded
 - Appropriate error returned



Sample Requirement Package

Requirement Traceability Mapping:

Requirement	Design Element	Method
R1	AuthenticationService	login()



Sample Requirement Package

Database Schema

Since this requirement involves user lookup, we need:

Users Table:

Field	Type	Notes
user_id	String	Primary Key
email	String	Unique
password_hash	String	Hashed, not cleartext
account_locked	Boolean	Default to false
created_at	Timestamp	

Login Audit Table:

Field	Type	Notes
audit_id	UUID	Primary Key
user_id	String	Foreign Key
timestamp	Timestamp	
success	Boolean	



Sample Requirement Package

Requirements Flowchart (Trace): This should be small.

User



API Endpoint (/login)



AuthenticationService.login()



UserRepository.findByEmail()



Password Verification



TokenService.generateToken()



Return JWT

Total components involved:

- API Layer
- AuthenticationService
- UserRepository
- TokenService

Database



Sample Requirement Package

SRP?

- ✓ AuthService only handles authentication.

Cohesion?

- ✓ All methods related to login.

Coupling?

- ✓ Depends on interfaces.

Abstraction?

- ✓ Database not directly accessed.

Trace Size?

- ✓ Small and focused.



Sample Requirement Package

For ONE requirement, we showed:

- ✓ Requirement summary
- ✓ Design element
- ✓ UML diagram
- ✓ Method specification
- ✓ Preconditions
- ✓ Postconditions
- ✓ Schema
- ✓ Requirement mapping
- ✓ Flowchart

This is exactly what you should replicate for each requirement.



Next Week

Industry-Style Design Review



Assignment: Weekly Standup Reflection

<https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection>

(Individual Assignment)

Due Feb 26 (and every week thereafter)



Assignment: First Self-Assessment

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Participation/SelfAssessment>

(Individual Assignment)

Due March 12



Assignment: Detailed Design Report

<https://rrostock1.github.io/Ursinus-CS375/Project/Design>

(Group Assignment)

Due March 26



Assignment: The Overdose

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose>

(Individual Assignment)

Due Apr 23



Due Next Week (March 12, by Midnight)

- Weekly Standup Reflection
- First Self-Assessment